COMS W4995 Applied Machine Learning

Assignment 2: From Trees to Neural Networks

Gradient-boosted decision trees vs. a multi-layer perceptron on UCI Bank Marketing

Author: Mit Sheth | ms7556

Colab notebook: **[COLAB LINK MISSING - rebuild with --colab URL]**
(the same link is repeated at the end of this report)

Table 1. Page map: where each part of the assignment is addressed.

Assignment step	Report section	Pages
Step 1 – Data preparation (audit, cleaning, split, preprocessing)	2.1, 2.2	1–2
Step 2 – Gradient-boosted trees (XGBoost): setup, tuning, curves, importance	2.3, 3.1	2–3
Step 3 – Neural network (MLPClassifier): setup, tuning, loss curves	2.3, 3.2	2–4
Step 4 – Evaluation and comparison (test metrics, PR curves, confusion matrices, timing, discussion)	2.4, 3.3, 4	2–6
Step 5 – Report: Introduction, Methods, Results, Discussion, AI disclosure, references	1–6	1–7

Summary

- **Task.** Predict whether a client subscribes to a term deposit (y = yes, 11.27% of 41,188 calls) from information available *before* the call. The leaking *duration* column is removed.
- **Selected models** (by validation average precision, AP): XGBoost **G7** (learning rate 0.01, depth 4, subsample 0.8, 863 trees; validation AP 0.4923) and MLP **M6** (one hidden layer of 64 units, learning rate 1e-4, L2 alpha 0.01, 135 epochs; validation AP 0.4825).
- **Test set** (6,179 rows, used once): AP 0.4819 (GBDT) vs. 0.4771 (MLP), against a no-skill baseline of 0.113. The two models are effectively tied; at the 0.5 threshold both are conservative (recall 0.24 vs. 0.25).
- **Cost and robustness.** GBDT fits about 2.8× faster (10.0 s vs. 27.8 s median, same 2-core Colab CPU) and was stable across all eight configurations (validation AP 0.482–0.492). The MLP ranged from 0.318 to 0.483 and overfitted badly at default settings until the learning rate was reduced and L2 regularization added.

1. Introduction

This report compares gradient-boosted decision trees (GBDT, XGBoost) and a multi-layer perceptron (MLP, scikit-learn *MLPClassifier*) on one tabular classification task. Both receive the same rows, split and features, so differences come from the model family and its tuning, not from the data pipeline.

Dataset. The UCI Bank Marketing dataset (*bank-additional-full.csv*) [1, 2] records 41,188 phone calls from a Portuguese bank's direct-marketing campaigns (May 2008 – November 2010), with 20 predictors and the binary target y. There are 10 numeric predictors (age, campaign, pdays, previous, duration and five macroeconomic indicators: emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed) and 10 nominal ones (job, marital, education, default, housing, loan, contact, month, day_of_week, poutcome). The positive class is rare (4,640 calls, 11.27%), so the primary metric is average precision (AP, area under the precision–recall curve), which reflects how well positives are ranked; accuracy alone would reward predicting "no" for everyone (88.7%).

2. Methods

2.1 Data audit and cleaning (Step 1a)

The audit checked every column's type and cardinality, missing values, sentinel codes, duplicates, rare levels and the target's domain. All cleaning rules are deterministic row-wise transformations (no statistics are learned), so they are applied before the split without leaking information from validation or test rows.

Table 2. Data-quality findings and the decision taken for each.

Issue	Finding	Decision
duration	Call length in seconds; only known after the call ends.	Dropped: target leakage for a pre-call prediction.
Missing values	0 pandas nulls, but the literal category "unknown" appears in 6 columns: default 8,597 (20.9%), education 1,731, housing 990, loan 990, job 330, marital 80.	Kept as its own category. On training rows it is informative: subscription is 5.3% when *default* is unknown vs. 12.9% when known.
pdays = 999	Sentinel for "no previous day count" in 39,673 rows (96.3%); real values are 0–27. 4,110 of these rows still have *previous* > 0 (poutcome = failure).	Added flag *pdays_recorded* (1 = a day count exists) and set the sentinel to −1 so it is not read as "999 days"; *previous* and *poutcome* are kept.
Duplicates	12 exact duplicate rows; 1,784 after dropping *duration*.	Kept: there is no client ID, so identical feature rows are plausibly different calls.
Rare levels	*default* = yes occurs in only 3 rows.	Kept; the encoder ignores categories unseen in training.
Target	Only "no"/"yes".	Mapped to 0/1; any other value raises an error.

2.2 Split and preprocessing (Steps 1b–1d)

A two-stage stratified split (random_state = 42) produced **train 28,831 / validation 6,178 / test 6,179** rows (70/15/15), with 3,248 and 696 positives in train and validation (11.3% each); test labels stay unread until the final evaluation (Section 3.3). Preprocessing is a column transformer fitted on training rows only: a one-hot encoder (categories unseen in training are ignored) expands the 10 nominal columns to 53 indicators, and the 10 numeric columns (including *pdays_recorded*) pass through, giving **63 features for both models**. The only difference is that the MLP's numeric columns are standardized with a standard scaler fitted on training rows; trees split on value order and need no scaling. Validation and test rows are only transformed with the training-fitted preprocessors, and the test set was not scored until both final models were fixed.

2.3 Model setup and tuning (Steps 2 and 3)

Table 3. Fixed settings and controlled experiments for both model families (8 configurations each).

Aspect	GBDT (XGBoost XGBClassifier) [3]	MLP (scikit-learn MLPClassifier) [4]
Input	63 features; numeric columns unscaled	Same 63 features; numeric columns standardized
Fixed settings	Histogram tree method (*tree_method* = hist); seed 42; other settings at defaults	Adam [5] solver, ReLU activation, batch size 200; seed 42; other settings at defaults
Training budget	Up to 5,000 trees, a safety cap only: an earlier 3,000 cap was hit by the depth-2 run, so it was raised; no run reaches 5,000	Up to 200 epochs (*max_iter*); a run ends earlier when training loss improves by less than 1e-4 for 10 consecutive epochs
Stopping and monitoring	Binary log loss on training and validation sets, validation listed last; early stopping after 50 rounds without validation improvement; predictions use trees up to *best_iteration*	Built-in early stopping off, so all training rows update the weights; *ConvergenceWarning*s are captured and recorded per run, not silenced
Phase A	Learning rate 0.01, 0.1, 0.3 (depth 4)	Hidden layers (64), (128, 64), (128, 64, 32) (learning rate 1e-3, alpha 1e-4)
Phase B	Max depth 2, 4, 6, 8	Initial learning rate 1e-4, 1e-3, 1e-2
Phase C (regularization)	Row subsample 1.0, 0.8, 0.6	L2 penalty alpha 1e-4, 1e-2, 1e-1, 1.0

Each phase changes one setting and carries forward the best value of the previous phase, so phases share a reference run. Within an XGBoost run, validation log loss decides when to stop adding trees; across runs, validation AP selects the configuration. For the MLP, *max_iter* is a fixed training budget, identical for every run, and the shared external validation set (not an internal holdout) is used only to compare configurations.

2.4 Selection, evaluation and timing (Step 4)

Each family's final configuration is the one with the highest validation AP (computed from predicted probabilities, unrounded). The test set is then scored once. AP is threshold-free; accuracy, precision, recall and F1 for the positive class use the default 0.5 probability threshold, which was deliberately not tuned. For timing, each selected configuration is refitted three times on the same Colab runtime (Linux x86_64, Intel Xeon 2.20 GHz, 2 CPU cores, no GPU; Python 3.13.18, pandas 2.2.3, NumPy 2.4.6, scikit-learn 1.6.1, XGBoost 3.4.1, matplotlib 3.10.0; seed 42) and the median is reported. Every refit is checked to reproduce the selected model, and preprocessing time (fit on train, then transform all splits) is measured separately.

3. Results

3.1 GBDT results (Step 2)

Table 4 lists all eight GBDT runs. **Learning rate** mainly changes speed, not the end point: 0.01 needs about 1,300 trees and 0.3 needs only 50, yet validation AP stays within 0.0024. Larger steps reach the validation minimum in fewer rounds and start overfitting sooner, so early stopping halts them earlier. **Depth** shows the bias–variance trade-off directly: depth 2 has the highest validation loss and needs over 3,000 trees (underfitting), while depth 8 fits training data much better (train log loss 0.235) without improving validation, widening the gap from 0.007 to 0.031. **Row subsampling** of 0.8 added slight regularization and gave the best validation AP (G7). Its training and validation curves (Figure 1) stay close together; early stopping keeps the first 863 trees (training ran to 913), at the validation minimum of 0.266.

Table 4. GBDT configurations. Trees = best_iteration + 1; gap = validation − train log loss. Selected row highlighted.

Config	Phase	Learning rate	Depth	Subsample	Trees	Train LL	Valid LL	Gap	Valid AP
G1	A	0.01	4	1.0	1,293	0.2596	0.2665	0.0069	0.4906
G2	A	0.10	4	1.0	182	0.2538	0.2662	0.0124	0.4882
G3	A	0.30	4	1.0	50	0.2550	0.2667	0.0117	0.4894
G4	B	0.01	2	1.0	3,137	0.2683	0.2678	−0.0005	0.4822
G5	B	0.01	6	1.0	429	0.2539	0.2657	0.0118	0.4898
G6	B	0.01	8	1.0	412	0.2352	0.2665	0.0313	0.4828
G7	C	0.01	4	0.8	863	0.2605	0.2660	0.0054	**0.4923**
G8	C	0.01	4	0.6	898	0.2596	0.2656	0.0060	0.4917

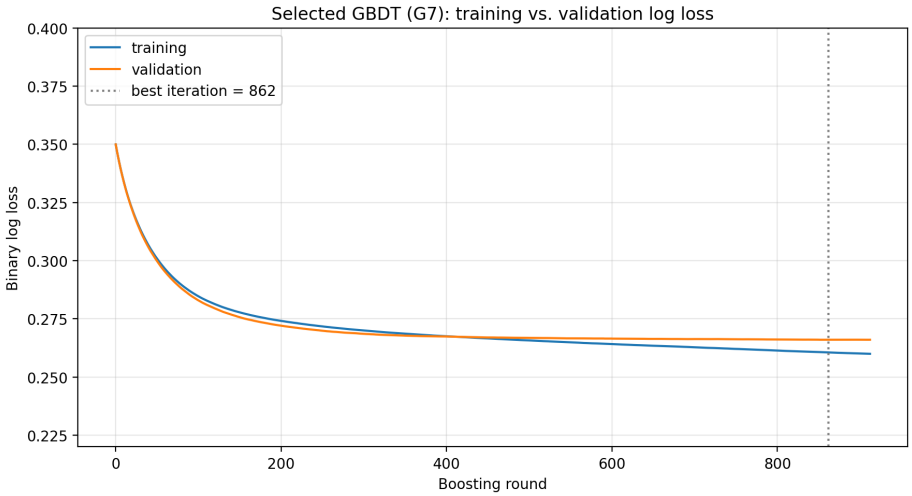


Figure 1. Training vs. validation log loss per boosting round for the selected GBDT (G7); the dotted line marks the early-stopping best iteration (0-based index 862, i.e. 863 trees).

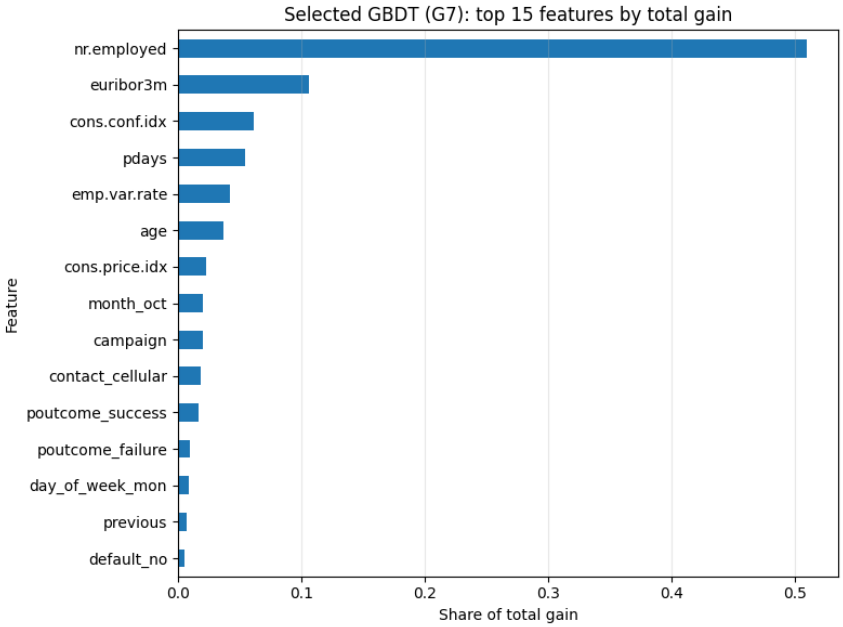


Figure 2. Top 15 features of the selected GBDT by share of total gain.

Feature importance (Figure 2) is measured as each feature's share of **total split gain** over the trees used for prediction (through *best_iteration*).

Macroeconomic context dominates: nr.employed alone accounts for about half of the gain, followed by euribor3m and cons.conf.idx. Among client and campaign features, the most important are pdays (recency of the last contact), age, the month, the number of calls this campaign, contact type and a previous success.

These scores describe association, not causation (Section 4.2).

3.2 MLP results (Step 3)

Table 5. MLP configurations ("Conv." = finished before max_iter without a ConvergenceWarning). Selected row highlighted.

Config	Phase	Hidden layers	LR	Alpha	Epochs	Conv.	Train LL	Valid LL	Gap	Valid AP
M1	A	(64)	1e-3	1e-4	200	No	0.2159	0.3065	0.0907	0.4099
M2	A	(128, 64)	1e-3	1e-4	200	No	0.0440	0.9191	0.8750	0.3180
M3	A	(128, 64, 32)	1e-3	1e-4	150	Yes	0.0504	1.1277	1.0773	0.3505
M4	B	(64)	1e-4	1e-4	200	No	0.2606	0.2688	0.0082	0.4805
M5	B	(64)	1e-2	1e-4	200	No	0.1824	0.4753	0.2928	0.3525
M6	C	(64)	1e-4	1e-2	135	Yes	0.2656	0.2684	0.0028	**0.4825**
M7	C	(64)	1e-4	1e-1	135	Yes	0.2687	0.2691	0.0004	0.4810
M8	C	(64)	1e-4	1.0	87	Yes	0.2780	0.2745	−0.0035	0.4672

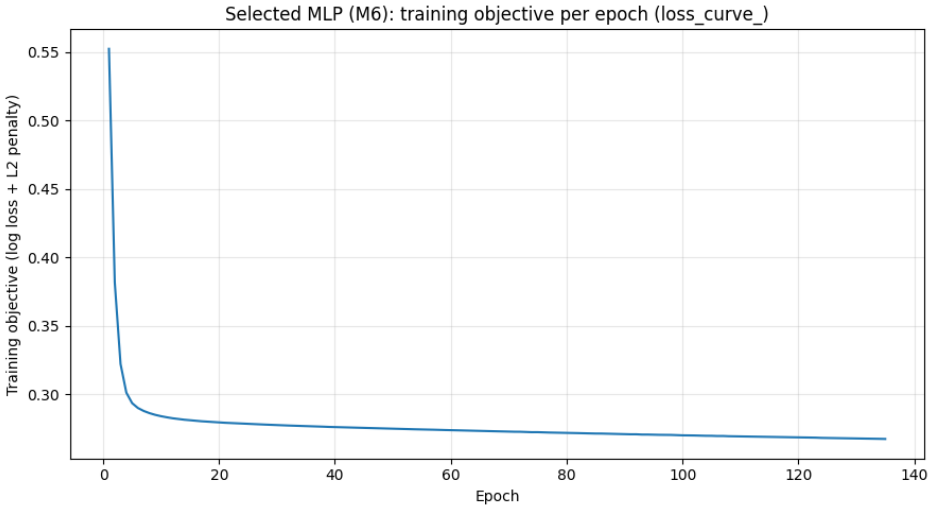


Figure 3. Training objective per epoch (loss_curve_, i.e. log loss plus L2 penalty) of the selected MLP (M6).

At the default learning rate, **larger networks memorize the training set**: the two-layer and three-layer models reach a training log loss of about 0.05 while validation loss rises above 0.9, and validation AP falls to 0.32–0.35 (Table 5, rows M2–M3). One layer of 64 units is the least overfit, so it was carried forward. Lowering the **learning rate** to 1e-4 nearly closed the gap (0.008) and raised AP to 0.48. At 1e-2 the training loss fell lowest of the three, but validation loss rose to 0.475: faster optimization only

meant faster overfitting. **L2 regularization** then trades variance for bias: alpha = 0.01 (M6) gave the best validation AP and converged in 135 epochs, while alpha = 1.0 underfits slightly (validation loss below training loss, AP 0.467). The selected model's training objective (Figure 3, including the L2 penalty) falls steeply, then plateaus. Four runs used all 200 epochs and raised ConvergenceWarnings, which are recorded in Table 5. These warnings mean the optimizer had not finished, not that training failed, and the selected model did converge.

3.3 Final comparison on the test set (Step 4)

Table 6. Selected models on the held-out test set (6,179 rows, 696 positives, prevalence 0.1126). Threshold 0.5 for the threshold-based metrics; times are medians of three refits (range in parentheses).

Metric	GBDT (XGBoost G7)	MLP (M6)
Validation AP (selection)	0.4923	0.4825
Test AP	**0.4819**	**0.4771**
Accuracy	0.9022	0.9003
Precision (positive class)	0.6917	0.6460
Recall (positive class)	0.2385	0.2543
F1 (positive class)	0.3547	0.3649
Predicted positives / true positives found	240 / 166	274 / 177
Training time, median (range)	9.994 s (7.278–10.046)	27.818 s (27.631–27.993)
Preprocessing time (median)	0.141 s	0.146 s

The two models are **practically tied** in ranking quality: test AP differs by 0.005, which is small compared with the sampling noise of a test set containing 696 positives. Both models are far above the no-skill AP of 0.113, and their PR curves nearly overlap across the whole recall range (Figure 4). At the 0.5 threshold both models are conservative, flagging only 240–274 calls and finding about a quarter of subscribers (Figure 5). The MLP trades a little precision for a little recall, which gives it a marginally higher F1. Accuracy (about 0.90) is only 1.3–1.5 points above always predicting "no" (0.887), so it is not informative here. In cost, GBDT trains **about 2.8× faster** on identical hardware, and preprocessing is negligible for both (about 0.14 s).

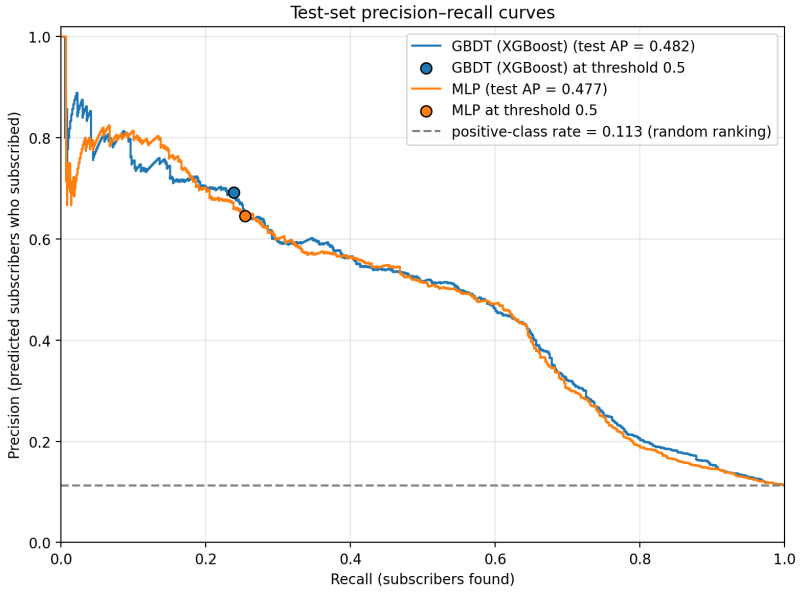


Figure 4. Test-set precision–recall curves with AP in the legend; dots mark the 0.5 threshold and the dashed line is the test positive-class proportion (0.113), the no-skill reference.

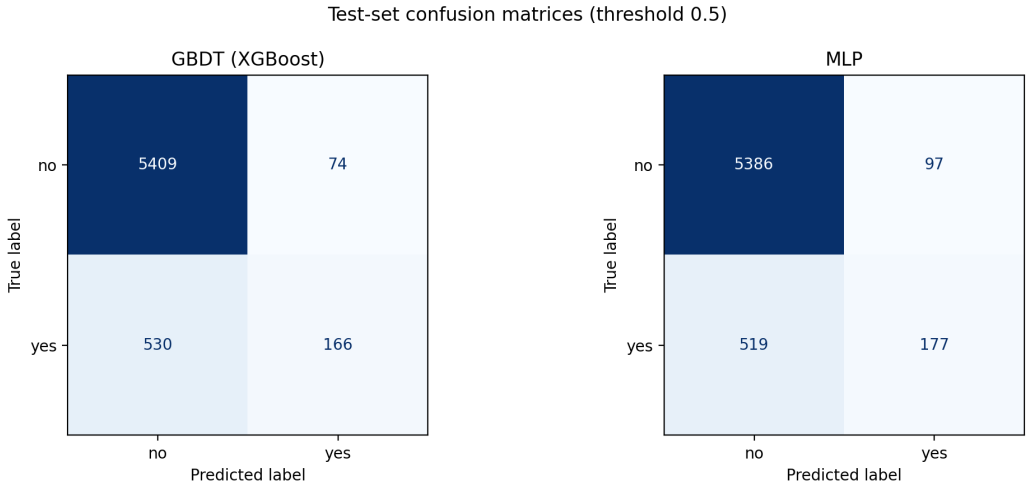


Figure 5. Test-set confusion matrices at threshold 0.5 (rows = true class, columns = predicted).

4. Discussion

4.1 Bias and variance

For GBDT, early stopping limits variance automatically. Shallow trees (depth 2) show bias, with higher loss on both sets, and deep trees (depth 8) show variance, with a gap about four times larger and no validation gain. The MLP's main problem was **variance**: with nothing restraining it, extra capacity memorized the training rows (training loss 0.04 vs. validation loss 0.92). A smaller step size and L2 regularization fixed this, and too much regularization (alpha 1.0) moved it back toward bias. Both selected models end with very small gaps (0.005 and 0.003), so their remaining error is mostly bias or irreducible noise. Without call duration, the pre-call features carry limited signal, and both families reach roughly the same ceiling.

4.2 When to prefer each model; sensitivity to tuning and scaling

On medium-sized tabular data with mixed types, **GBDT is the better default**. It matched or beat the MLP, trained 2.8× faster, needed no scaling, and its eight configurations all landed within about 0.01 AP of each other (0.482–0.492). That robustness means a reasonable model is obtained with little tuning. The MLP was **highly sensitive to hyperparameters** (validation AP ranged from 0.32 to 0.48) and depends on feature scaling: unscaled inputs such as nr.employed (≈ 5,000) would dominate the gradients of binary indicators, whereas trees are invariant to monotone rescaling. An MLP becomes preferable for large or unstructured data (text, images, sequences), when learned embeddings or joint training with other neural components are needed, or when predictions must vary smoothly with the inputs.

Categorical and missing values. Here both models used identical one-hot features, and "unknown" was kept as an explicit category because it carries signal (for example, the lower subscription rate when *default* is unknown). In general, trees handle such data more naturally: XGBoost can route true missing values (NaN) down a learned default branch and also supports native categorical splits. An MLP needs every input to be numeric and complete, which requires explicit imputation, encoding and scaling, and sentinel values such as *pdays* = 999 must be recoded, as was done here.

Interpretability. GBDT provides built-in importance scores, but these describe **association, not causation**. The macroeconomic indicators are strongly correlated with each other and with calendar time, so the gain shared between them is partly arbitrary. The model mainly learned *when* campaigns succeeded, which does not show that employment levels cause subscriptions. The MLP has no comparable built-in measure; model-agnostic methods such as permutation importance or SHAP could be applied to either model, and would carry the same caveat.

4.3 Limitations

- **Single split and seed.** One stratified split with seed 42; differences of about 0.005 AP are within noise, so repeated or cross-validated splits would be needed to claim a winner.
- **Limited search.** Tuning was sequential and one factor at a time, with eight runs per model. The MLP architecture was chosen at the default learning rate and alpha, so a larger network might perform well once regularized.
- **Default threshold.** The 0.5 threshold gives low recall; a real campaign would choose a threshold from the precision–recall curve to fit the cost of a call versus the value of a subscription.
- **Historical benchmark.** The data come from 2008–2010 campaigns. A random split mixes time periods, so it does not test performance on future campaigns. The dominance of macroeconomic features suggests that a time-ordered split would score lower.

- **Timing.** Times come from a shared 2-core Colab CPU and are indicative only; the ratio between the models is more reliable than the absolute seconds.

5. AI Tool Usage Disclosure

5.1 AI tools used

- ChatGPT (OpenAI)
- Cursor
- CodeRabbit

5.2 What the AI tools contributed

- Generated and refined starter and final Python code for:
  - Data loading, cleaning, stratified splitting and leakage-safe preprocessing
  - GBDT and MLP training, tuning, evaluation, plots and timing
- Assisted with implementing and debugging:
  - The leakage-safe train/validation/test split and preprocessing workflow
  - XGBoost early stopping and MLP convergence / regularization
  - Validation-AP selection, test metrics, PR curves, confusion matrices and timing
- Provided conceptual explanations of:
  - Target leakage (*duration*), sentinel values (*pdays* = 999) and average precision
  - Bias–variance under capacity/regularization, and total-gain importance as association not causation
- Assisted with debugging and consistency checks to identify potential implementation or interpretation issues.
- Assisted with drafting and refining portions of the report structure, tables, figure descriptions and written analysis.
- CodeRabbit was used for limited automated PR/code reviews to identify potential issues and inconsistencies during development.

5.3 Self contribution

- Selected the UCI Bank Marketing dataset and established the overall workflow according to the assignment requirements.
- Defined the methodological constraints used throughout the analysis, including:
  - Avoiding data leakage and using a consistent train/validation/test split for both models
  - Keeping the test set untouched until both configurations were fixed
  - Following the requirements specified in the assignment
- Reviewed the proposed preprocessing and modeling approaches and made/approved the final methodological decisions, including:
  - Dropping *duration*, recoding *pdays* = 999, and keeping literal *unknown*
  - MLP-only scaling and the selected configurations (G7, M6)
- Ran the notebook in Google Colab and inspected the resulting outputs, tables and visualizations.
- Verified that the implemented workflow and results were consistent with the actual dataset and written interpretations.
- Evaluated and validated the analysis, model comparisons, visualizations and conclusions, and reviewed/refined the AI-assisted code and explanations throughout development.

6. References

[1] S. Moro, P. Cortez and P. Rita. A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems*, 62:22–31, 2014.

[2] UCI Machine Learning Repository: Bank Marketing. https://archive.ics.uci.edu/dataset/222/bank+marketing

[3] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. *KDD*, 2016; documentation at https://xgboost.readthedocs.io

[4] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR* 12:2825–2830, 2011; MLPClassifier and average_precision_score documentation at https://scikit-learn.org

[5] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *ICLR*, 2015.

7. Additional Resource

Github Link: https://github.com/2004Luci/AML

Colab notebook: **[COLAB LINK MISSING - rebuild with --colab URL]**